

Glance — Engineering & Compliance Audit

June 2026

Glance — Engineering & Compliance Audit

Date: June 2026 · **Version audited:** post-M4 (main) · **Auditors:** code/architecture review, data-protection review, dependency & hardening review.

This document is the working backlog. Each finding has an ID, a severity, and a remediation tag: **[FIX NOW]** (no new infra), **[PRE-DEPLOY]** (needs auth / database / infra), **[LEGAL]** (needs a qualified solicitor), **[SCALE]** (needed for horizontal scale). Compliance guidance here is informed but **not legal advice**.

Executive summary

The **domain core is production-quality**. The salience engine, stats aggregator, session recorder, sentiment/toxicity analysers, and the platform/AI/storage *interfaces* are pure, strictly-typed, well-tested, and cleanly abstracted — the architecture’s seams genuinely deliver on “platform-agnostic.”

The **server is a single-process, single-session, in-memory prototype**. It has no authentication, no multi-tenancy, no WebSocket hardening, no graceful shutdown, an O(n) file store, and it stores third-party chat (names + message text) in plaintext with no retention limit. It is an excellent local app and a compelling demo. **It is not deployable as a multi-tenant SaaS** in its current shape, and it must not be exposed to the public internet until the Critical items below are resolved.

The path is clear and the foundation is sound: this is a transport/ownership/ persistence hardening effort, not a rewrite of the product.

Scorecard

Dimension	Grade	Headline
Domain logic & correctness	A–	Pure, tested, deterministic; minor ID/randomness nits.
Architecture & platform-agnosticism	A–	Clean swappable seams; minor hardcoded localhost/twitch.
Test coverage	C+	Core well-tested; server, adapters, AI untested .
Security	D	No auth, CORS *, no WS origin/heartbeat/limits.

Scalability	D	Single process, single session, in-memory, O(n) store.
Data protection / compliance	D	Indefinite plaintext PII; no retention/erasure; AI sub-processor undisclosed.
Dependency currency	B-	Mostly current installs, but Vite floor is CVE-vulnerable ; majors behind.
Observability / ops	D	Silent catch{ } everywhere; no metrics; static /health.

CRITICAL — blockers before any public/multi-tenant exposure

C1 — No authentication or authorization on the control + data plane. [PRE-DEPLOY]

Every REST endpoint (POST/DELETE /api/session, PUT /api/settings, GET/DELETE /api/sessions/:id) and the WebSocket are fully open; CORS: '*'; no WS Origin check. Anyone who can reach the port can hijack the session, change settings, **read/delete every archived session (chat names + text)**, and any website the streamer visits can drive their server (CSWSH/CSRF). *Fix*: per-tenant auth (signed token on REST Authorization + validated on the WS upgrade), CORS origin allowlist, WS origin verification. The local-grade pieces (origin check, CORS allowlist, payload caps) are **[FIX NOW]**; the full token/identity system is **[PRE-DEPLOY]**.

C2 — Single global session; architecturally single-tenant. [SCALE]

One SessionController / GlanceEngine / timer set / state per process. A second creator connecting calls teardown() and **destroys the first creator's session**; the gateway broadcasts every message to *all* clients with no per-tenant routing. *Fix*: introduce a tenant/room key; key controllers, storage, settings and WS subscriptions by it; fan-out per room. This is the core scale-out work.

C3 — WebSocket server has no heartbeat, payload cap, backpressure, or rate limit.

[FIX NOW] No ping/pong → half-open TCP connections accumulate as zombie clients (memory/FD leak). broadcast() ignores client.bufferedAmount → one slow consumer makes Node buffer unboundedly (OOM/DoS). WebSocketServer has no maxPayload (defaults to 100 MB/frame). No per-IP connection cap. *Fix*: heartbeat + terminate-on-missed-pong; check bufferedAmount and drop slow clients; set maxPayload to a few KB; cap connections per IP. **Fixing this batch now.**

C4 — FileStorage.listSessions is an O(n) full scan on the event loop. [FIX NOW — PRE-DEPLOY] GET /api/sessions reads + JSON.parsees **every** session file (each up to 250 timeline entries) synchronously just to build summaries — called on every Replay mount and after every delete. At thousands of archives this blocks the single event loop and stalls the live pipeline for all creators. *Fix now*: maintain a lightweight summary index file;

never read full bodies to list. *Pre-deploy*: move to SQLite/Postgres (the Storage seam already anticipates this).

C5 — No graceful shutdown; in-flight session archive is lost on every deploy. [FIX NOW] shutdown() calls process.exit(0) immediately, but persist() is fire-and-forget async (AI recap + saveSession). The exit races the unresolved promise → the recording session's archive is silently dropped; WS clients are cut without a close frame. *Fix*: async shutdown that awaits outstanding persistence (with a timeout), closes the WS server with a grace period, then exits. **Fixing this batch now.**

C6 — Vite version floor permits four 2025 path-traversal CVEs. [FIX NOW] vite ^5.4.11 allows installs below the fixes for **CVE-2025-30208 / -31125 / -31486 / -32395** (server.fs / @fs arbitrary file read; fixed at **5.4.18**). Dev-server-only and requires --host, but the pin is a real liability (installed build happens to be patched; the floor is not). *Fix*: raise the floor to >=5.4.18 immediately (then plan the Vite 6/7 major). **Fixing this batch now.**

C7 — Indefinite plaintext retention of third-party chat PII; no erasure path. [FIX NOW + LEGAL] Archived SessionDetail JSON stores chatter **display names + verbatim message text** forever, unencrypted, with deletion only by whole session. This violates storage-limitation/minimisation duties and Twitch's developer terms, and cannot honour an erasure request for one chatter. See **Compliance** below. *Fix now*: retention TTL/sweep + channel/per-author deletion + an opt-out of storing raw text. *Legal*: lawful basis, notice, DPIA. **Retention plumbing is Fix Batch 3.**

C8 — Storing under-13 chatters' data triggers COPPA you cannot satisfy. [LEGAL] Twitch has under-13 users; a stored message revealing a minor can constitute "actual knowledge," triggering verifiable-parental-consent and the 2025 COPPA retention rule (\$312.10) — neither satisfiable for anonymous chatters. Penalties are per-record. *Fix*: "never knowingly store child data" posture + detect-and-purge + the written retention policy; **US privacy counsel required.**

HIGH

H1 — Error-swallowing everywhere. [FIX NOW] Pervasive empty catch {} (session.ts, engine.ts, anthropic.ts, storage.ts, settings-store.ts, config.ts, gateway.ts). AI/disk/archive/config failures vanish with no log or metric — you'd be blind in production. *Fix*: log every caught error with context; add counters. **This batch.**

H2 — Settings merge spreads unvalidated input. [FIX NOW] { ...this.current, ... (patch as object) } trusts arbitrary JSON before normalisation. Not a classic proto-pollution sink (object spread + field whitelist), but fragile and, combined with C1, allows unauthorized settings mutation. *Fix*: pass patch straight to normalizeEngineSettings; guard __proto__/constructor. **This batch.**

H3 — Twitch reconnect storm risk. [FIX NOW] Exponential backoff but **no jitter**, no max-attempt cap, and error+close can both fire → double reconnect timers. At scale a

Twitch blip produces a synchronized thundering herd (risking IP bans). *Fix*: full jitter + single-flight reconnect guard + cap/alert. **This batch.**

H4 — Unbounded session archives; chat PII retained forever. [FIX NOW] No TTL, no session cap; disk grows without bound. *Fix*: retention policy + eviction. **Fix Batch 3.**

H5 — No server / integration / adapter tests. [FIX NOW] Tests cover packages/core only. The riskiest code — gateway, session lifecycle, storage, settings-store, and the **IRC parser** (exported “for testing” but untested) — is unverified. *Fix*: add tests; gate CI. **Fix Batch 4.**

H6 — channel input not validated — outbound-connection abuse. [FIX NOW] Any string spins up a TwitchAdapter that reconnect-loops forever; an attacker (via C1) can open arbitrary outbound Twitch connections. *Fix*: validate `^[a-z0-9_]{3,25}$`, cap length, gate behind auth/ownership. **This batch.**

H7 — AI calls have no timeout, concurrency cap, or budget guard. [FIX NOW + SCALE] anthropic.ts awaits with no AbortController; per session a summary fires $\geq$ every 4 s and priorities every 9 s. At thousands of sessions this is unbounded paid fan-out; a slow response stalls the cycle. *Fix now*: per-call timeout. *Scale*: global concurrency limiter + per-tenant cost caps. **Timeout this batch.**

MEDIUM

- **M1 — Settings are global, not per-tenant.** Same root as C2. [SCALE]
 - **M2 — Skipped AI cycles are invisible; timers not unref'd.** Log skips; consider adaptive cadence. [FIX NOW]
 - **M3 — Math.random() IDs** (sessions/messages/summaries). Guessable + collision-prone once addressable under auth. *Fix*: crypto.randomUUID(). [FIX NOW — this batch]
 - **M4 — Silent id sanitisation in FileStorage** can collapse two ids to one filename (overwrite). *Fix*: validate-and-reject, or hash ids. [FIX NOW]
 - **M5 — No structured logging / metrics / real health.** /health is static {ok:true}. *Fix*: JSON logging + Prometheus metrics + /ready. [PRE-DEPLOY]
 - **M6 — Config not validated; no fail-fast at boot.** *Fix*: validate required env, fail fast. [FIX NOW]
 - **M7 — readJson rejects oversize but doesn't req.destroy().** Socket keeps receiving. *Fix*: destroy on reject; return 413. [FIX NOW — this batch]
-

LOW

- **L1** CORS allows DELETE but only content-type header — update when auth headers land.
- **L2** ESLint disables no-explicit-any; no-unused-vars only warn — loosens the strict-TS safety net.

- **L3** No Dockerfile / deploy manifest / LICENSE; engines.node >=20.11 while Node 20 is **EOL (Apr 2026)**.
 - **L4** Front-ends hardcode localhost (only the port is env-driven) — host must be configurable for deploy. **[FIX NOW]**
 - **L5** Front-end reconnect uses fixed 1500 ms (no backoff/jitter).
 - **L6** document.getElementById('root')! throws opaquely if markup changes.
-

Compliance & data protection (summary — see full review; not legal advice)

Chat **message text + display names are personal data** under UK/EU GDPR, CCPA, and COPPA. The moment Glance persists a message, the full weight applies. Five ranked risks:

1. **COPPA / under-13 data (existential).** **[LEGAL]** — can't obtain parental consent for anonymous chatters; 2025 retention rule; per-record penalties. → never-knowingly-store posture + detect-and-purge + written retention policy.
2. **Indefinite plaintext retention + no erasure.** **[FIX NOW + LEGAL]** — violates storage-limitation/minimisation (UK/EU GDPR Art 5) and Twitch DSA (chat cache exception is ~24h, no "public databases"). → retention sweep, per-author deletion, encryption at rest, anonymisation.
3. **Sending chat to Anthropic with no DPA/transfer/disclosure/Twitch-permission.** **[FIX NOW + LEGAL]** — Anthropic is a sub-processor (GDPR Art 28); UK/EU→US transfer needs SCCs/IDTA; Twitch DSA limits third-party sharing. → accept Anthropic Commercial Terms (DPA auto-applies), request **Zero Data Retention**, ensure no-training is on, disclose sub-processor, resolve Twitch permission.
4. **No lawful basis + no Art 14 transparency ("invisible processing").** **[LEGAL]** — realistic basis is legitimate interests (needs a documented LIA + DPIA); publish a privacy notice covering chatters.
5. **Controller-vs-processor misjudgement.** **[LEGAL]** — you're likely a *controller* for the AI/analytics ("moat") processing; that decides who owes chatters transparency.

Compliance checklist (local → SaaS): retention limits + sweep · granular (per-author/message) deletion + honour Twitch message-delete events · encrypt at rest + move off plaintext files · minor-safety detect-and-purge · Anthropic DPA/ZDR/no-training · privacy notice · LIA · DPIA · Art 30 records · controller decision + streamer DPAs · data-subject-rights mechanism. **Get a solicitor for the COPPA analysis, the legitimate-interests/Art 14 balancing, and the controller/processor characterisation.**

Dependency currency & upgrade plan (verified, mid-2026)

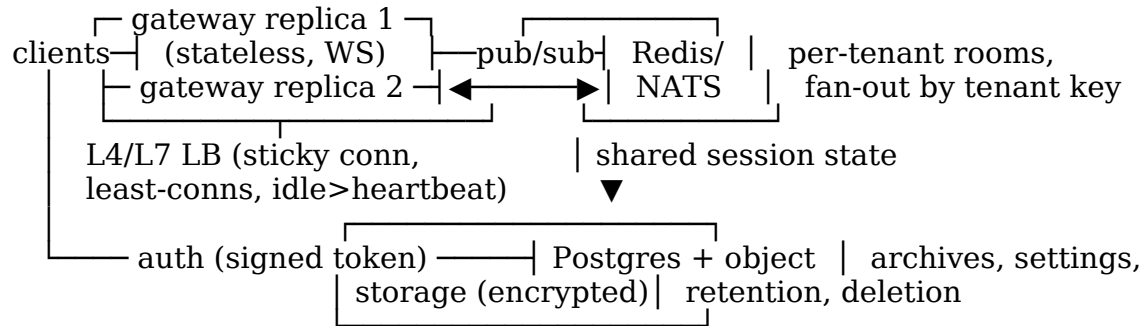
Installed builds already drift ahead of the pins; the **pin floors** are the risk.

Package	Pinned	Latest	Action	Risk
vite	^5.4.11	6 / 7 / 8	Floor → ≥5.4.18	High at major

			now (closes 4 CVEs); plan Vite 7 (needs Node $\geq 20.19/22.12$)	
node engine	≥ 20.11	24 LTS (20 EOL)	$\rightarrow \geq 22.12$, ideally ≥ 24; CI to Node 24	Low-mod
eslint	$\wedge 9.17$	10.5 (9.x EOL soon)	$\rightarrow \wedge 10$ (flat-config only; time-sensitive)	Low if flat
react / react-dom	$\wedge 18.3.1$	19.2	$\rightarrow 19$ (mature; codemods; drop forwardRef)	Moderate
typescript	$\wedge 5.6.3$	6.0	$\rightarrow \wedge 6$ (last JS-based TS; stricter)	Low-mod
vitest	$\wedge 2.1.8$	3.x	$\rightarrow \wedge 3$ (with Vite)	Moderate
pnpm	9.15	10 / 11	$\rightarrow 10$ (lifecycle scripts now opt-in via onlyBuiltDependencies)	Moderate
@anthropic-ai/sdk	$\wedge 0.32.1$	~ 0.105 (still 0.x)	upgrade incrementally (breaking 0.x minors; isolated in packages/ai)	High effort
ws	$\wedge 8.18.0$	8.x	routine bump (floor already past CVE-2024-37890)	Low
tsx / prettier / typescript-eslint	—	current majors	routine bumps	Low

Order: (1) Vite floor [security] $\rightarrow$ (2) Node engine + CI [support] $\rightarrow$ (3) ESLint 10 [EOL] $\rightarrow$ (4) React 19 / TS 6 / Vitest 3 / pnpm 10 / Anthropic [deliberate cluster] $\rightarrow$ (5) routine bumps. Each as a PR with pnpm verify green.

Target architecture for scale (the C2/M1/M5 rewrite)



Prerequisites for “thousands of creators”: per-tenant ownership; auth/identity; stateless gateways + Redis/NATS pub-sub fan-out; a real database with retention & deletion; WS heartbeat/backpressure/limits; structured logging + metrics; readiness probe + graceful drain; autoscale on connection count/memory.

What’s genuinely strong (don’t touch)

- **@glance/core** — pure, deterministic, the best-tested code in the repo; the salience “moat” lives here cleanly.
- **The three seams** — PlatformAdapter, AIProvider, Storage are real, swappable interfaces; no Twitch-isms leak into core; render-feed hooks are presentation-free. Kick/YouTube or OpenAI/local models slot in without touching the server, as designed.
- **Graceful AI degradation** — every AI path falls back to deterministic rules; the product works with zero keys.
- **CI gate** — typecheck + lint + test + build on every push is already in place.

Remediation roadmap

Now — [FIX NOW], no new infra (in progress): Batch 1 (security/correctness): WS heartbeat + maxPayload + origin check + backpressure + conn cap (C3); CORS origin allowlist (C1 partial); channel validation (H6); async graceful shutdown (C5); error logging (H1); crypto.randomUUID (M3); settings-merge hardening (H2); readJson destroy + 413 (M7); Twitch reconnect jitter/guard (H3); AI call timeout (H7); config fail-fast (M6); configurable front-end host (L4); listSessions summary index (C4 interim). Batch 2 (deps): Vite floor >=5.4.18 (C6); Node engine >=22.12 + CI Node 24; routine bumps. Batch 3 (data): retention TTL/sweep + cap + channel/per-author deletion + no-raw-text mode (C7/H4). Batch 4 (tests): IRC parser, storage, settings-store, prioritize seam (H5).

Pre-deploy — [PRE-DEPLOY], needs infra/auth/db: Per-tenant identity + auth (C1 full, C2); Postgres + object storage, encrypted (C4 full); structured logging + metrics + /ready + graceful drain (M5); AI concurrency limiter + cost caps (H7 scale).

Legal — [LEGAL], before public launch: COPPA analysis; lawful basis + LIA + DPIA + privacy notice; controller decision + streamer DPAs; Anthropic DPA/ZDR; Twitch data-sharing permission.

Scale — [SCALE]: Stateless gateways + Redis/NATS pub-sub fan-out; WS-aware load balancing; autoscale.